Processor Architecture

Computer Organization

Chandra Gummaluru

chandra-gummaluru.github.io



processor

the part of a computer that fetches instructions and carries them out

A processor has two parts that work together:

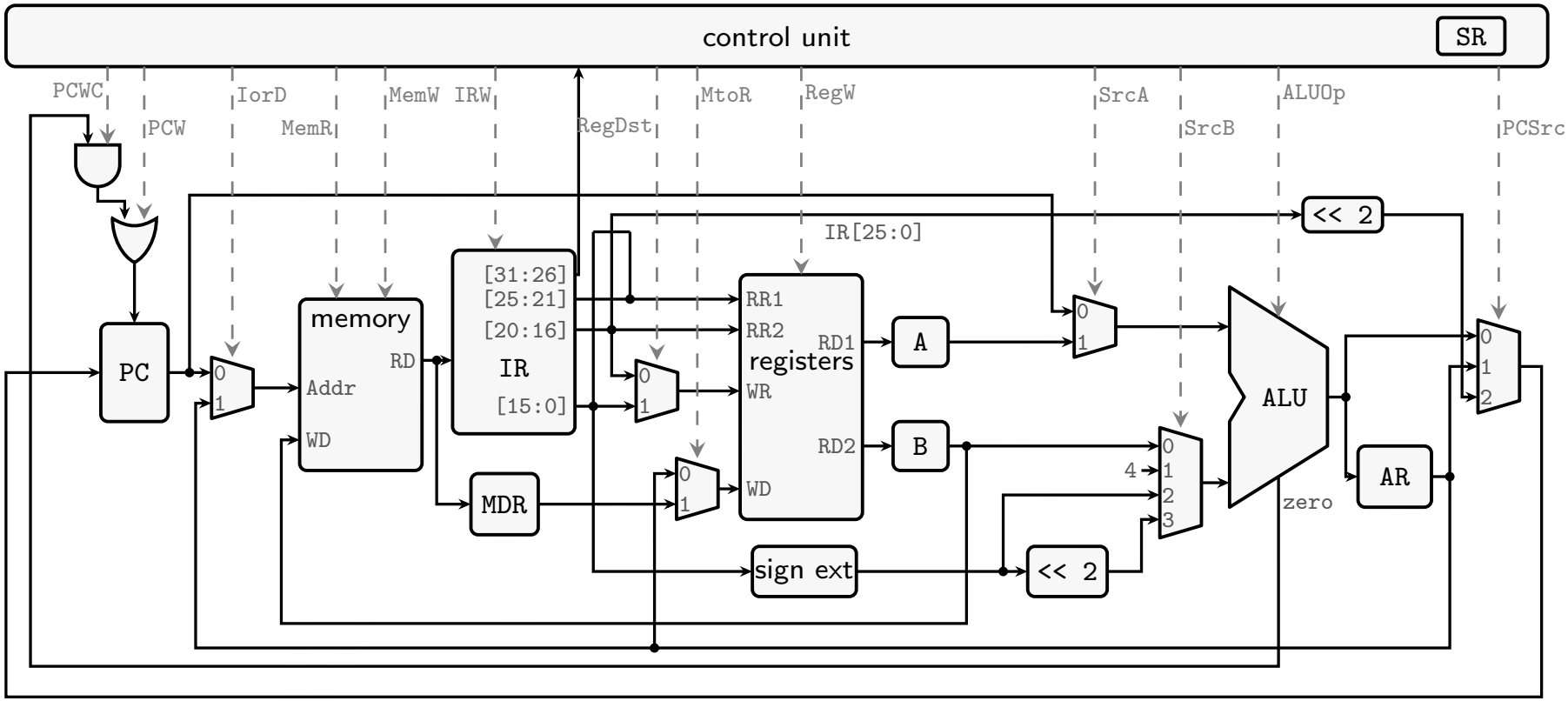
datapath

the components that store and transform data

control

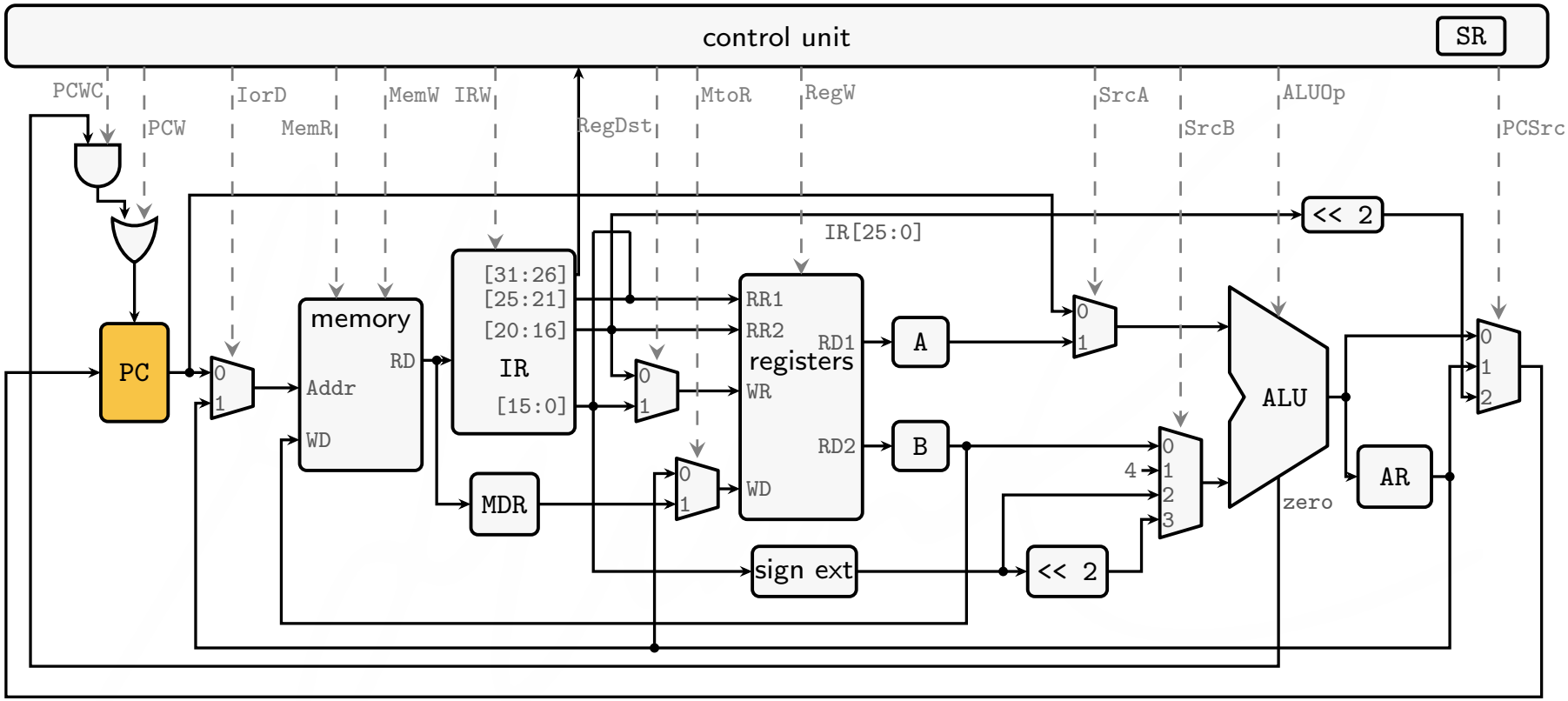
the logic that directs the datapath, step by step

Here is the whole computer. The control unit sits at the top and sets the control signals (dashed). Everything below it is the datapath, including memory.



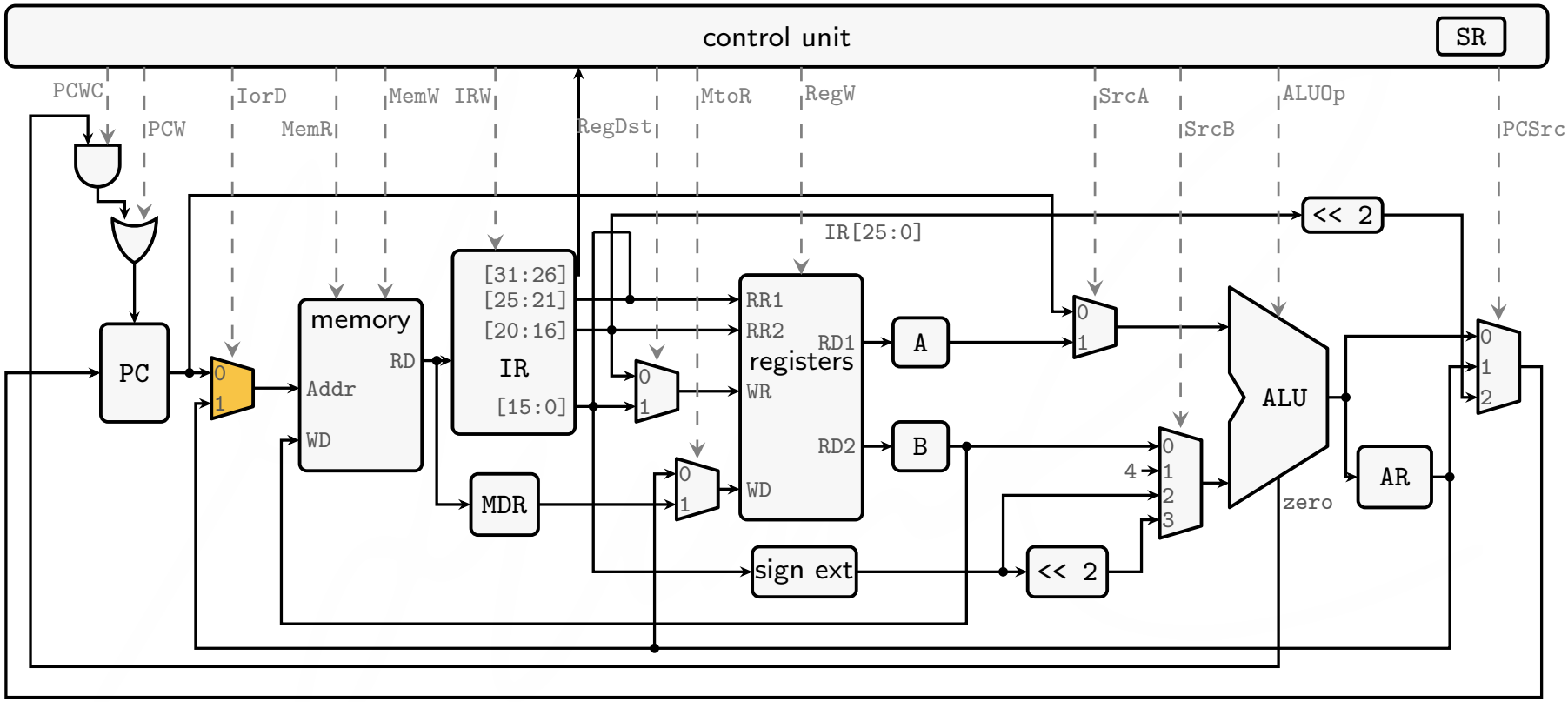
the datapath

1. The **program counter** (PC) holds the address of the next instruction.



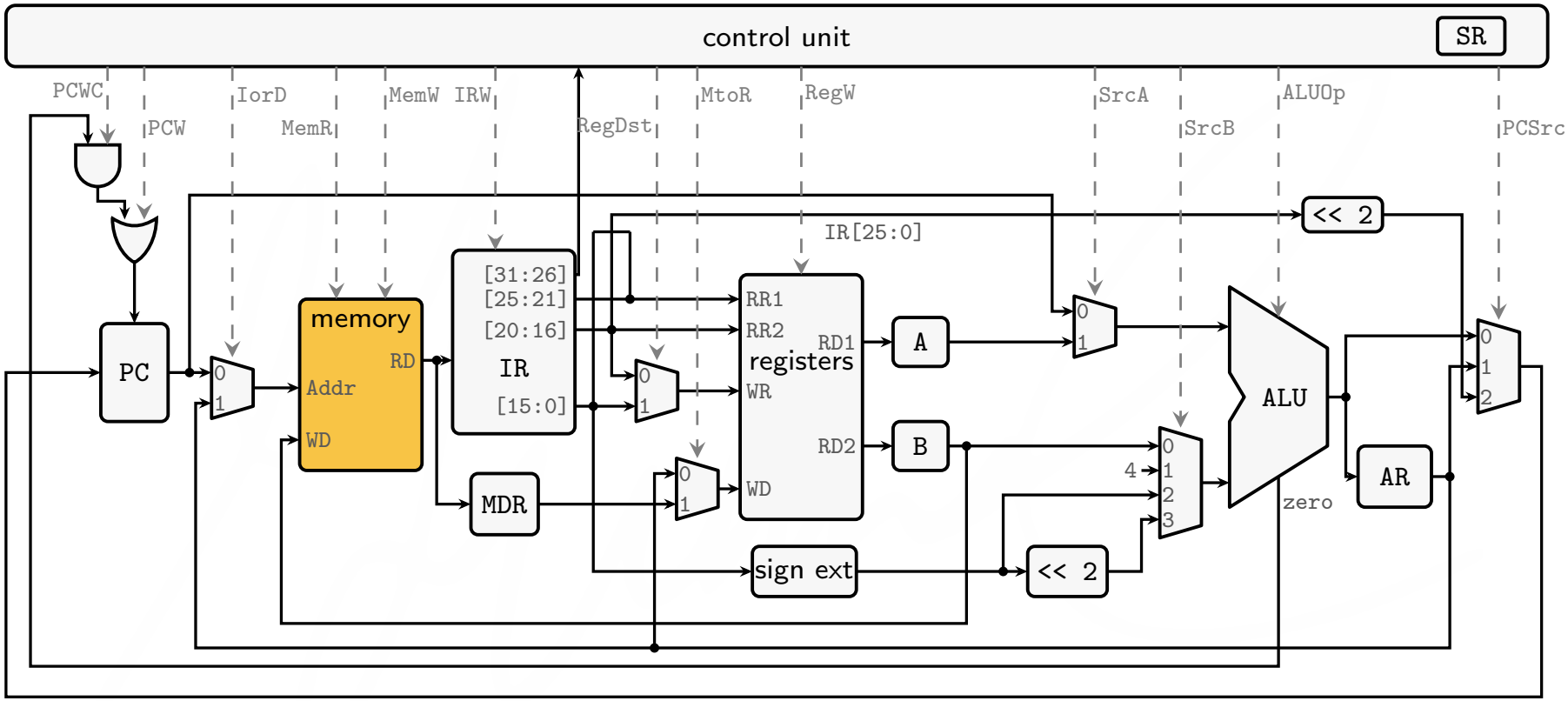
the PC is a register; it changes only when the gates above it say so (step 17)

2. A mux picks the address sent to memory.



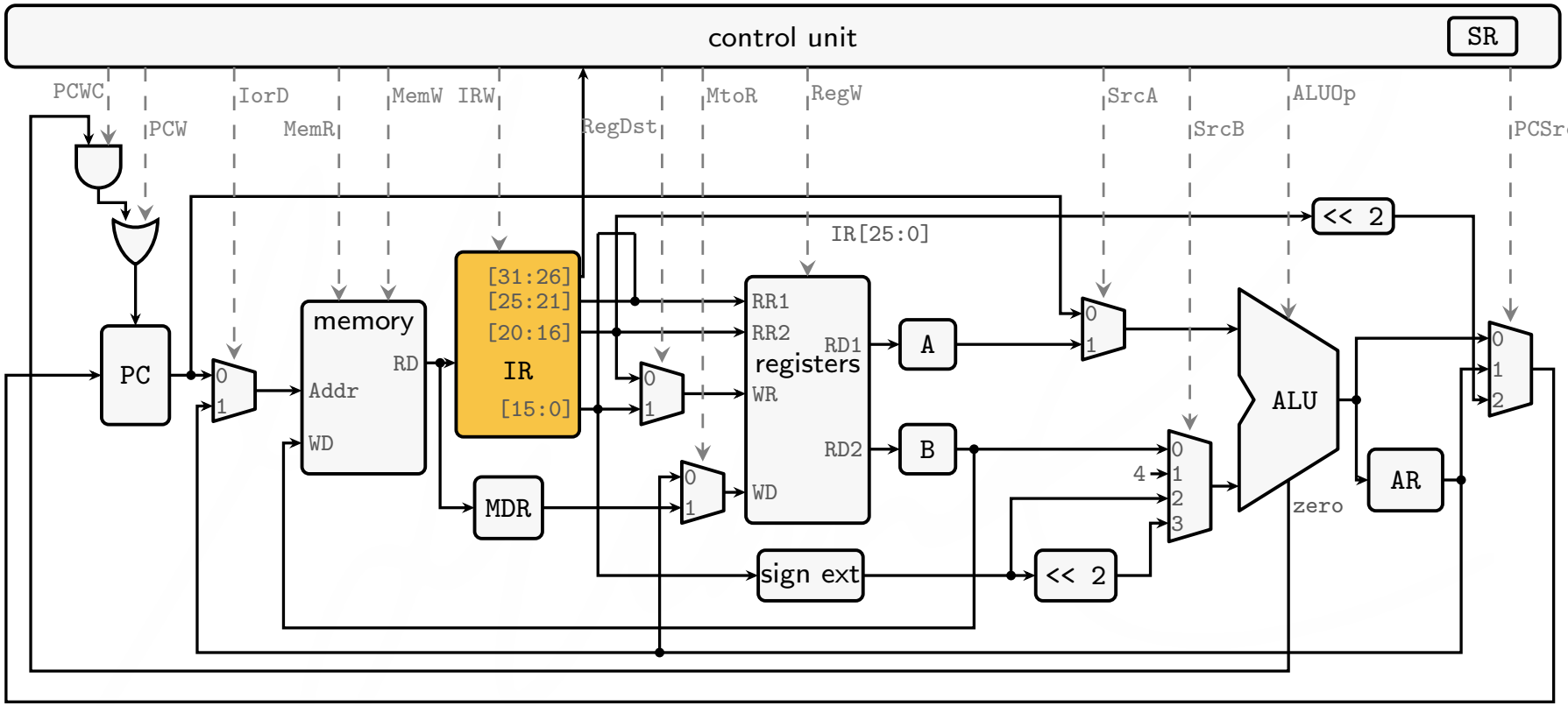
IorD picks the PC, to fetch an instruction, or AR, to load or store data

3. Memory reads the word at that address.



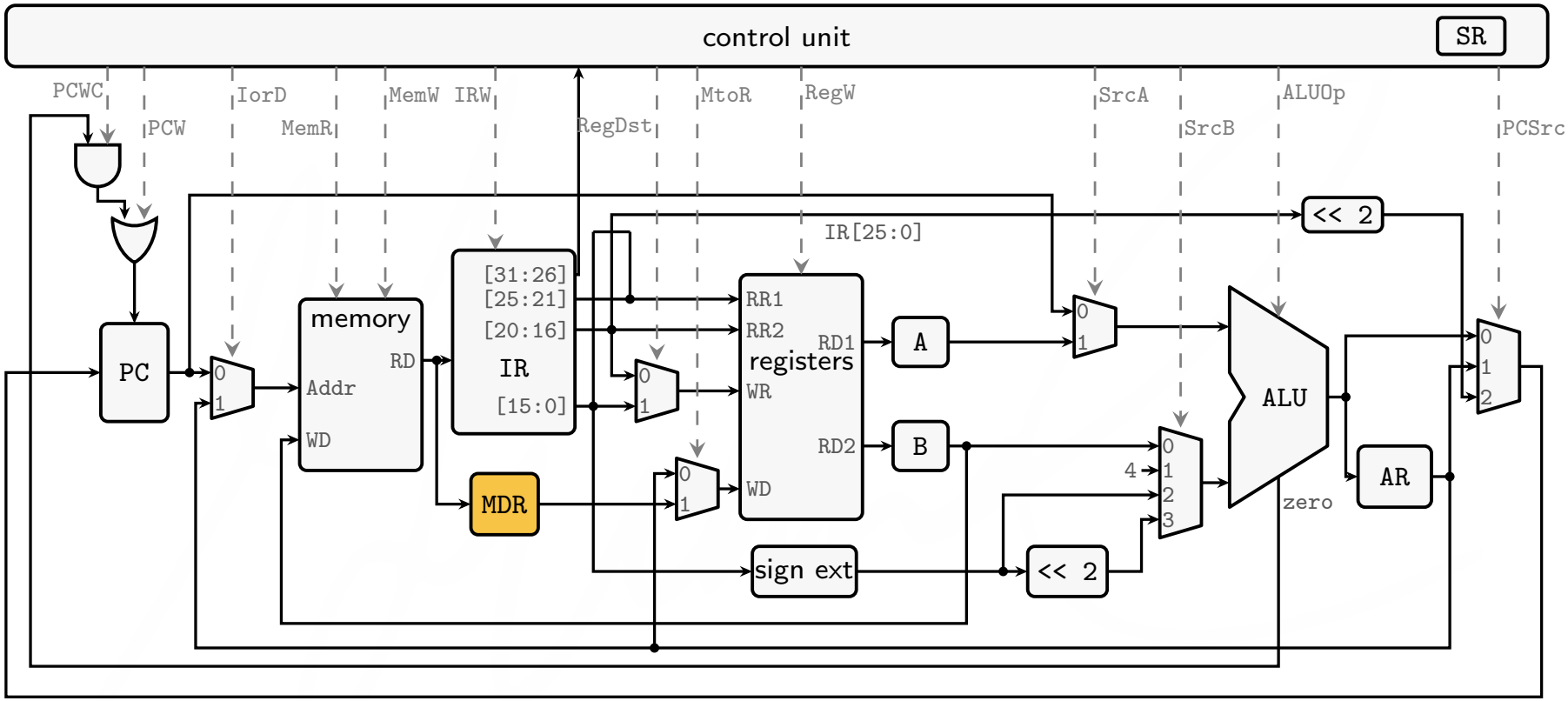
memory holds both the program and its data; it reads `Addr` when `MemR` is set, and writes `WD` there when `MemW` is set

4. If the word is an instruction, it is stored in the **instruction register** (IR).



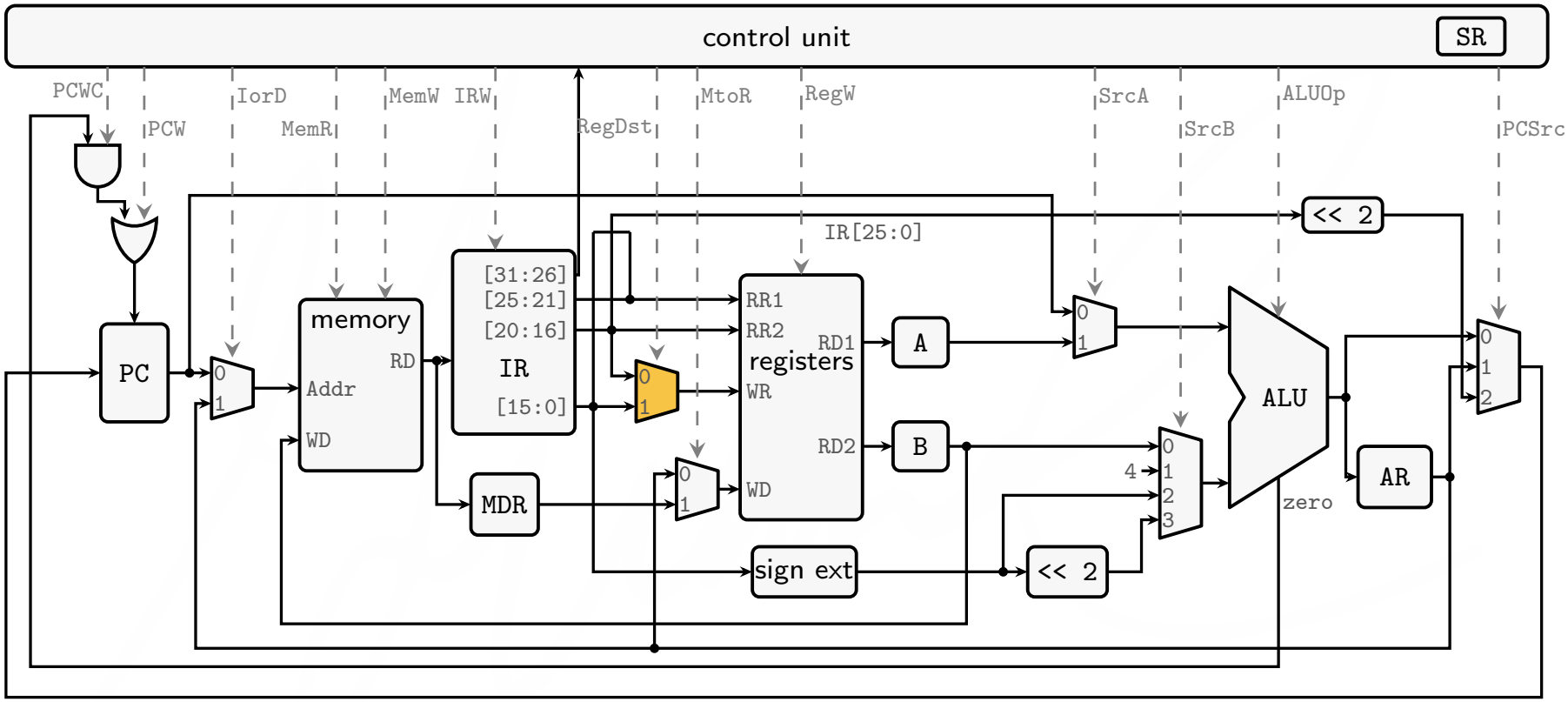
IRW loads it; its fields are the opcode IR[31:26], two registers IR[25:21] and IR[20:16], and a constant IR[15:0]

5. Otherwise, it is stored in the **memory data register** (MDR).



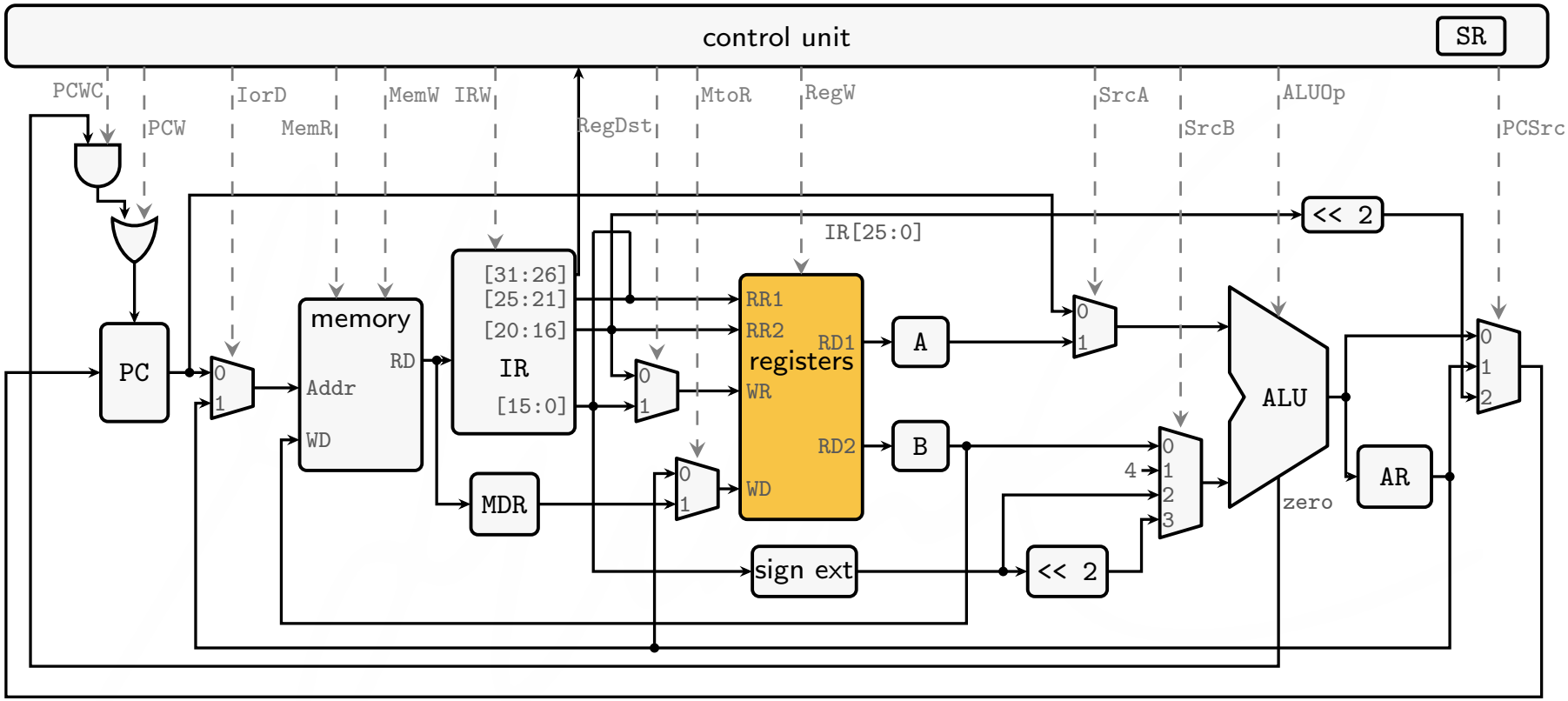
the loaded word waits in the MDR so it can be written to a register on the next cycle

6. A mux picks which register to write.



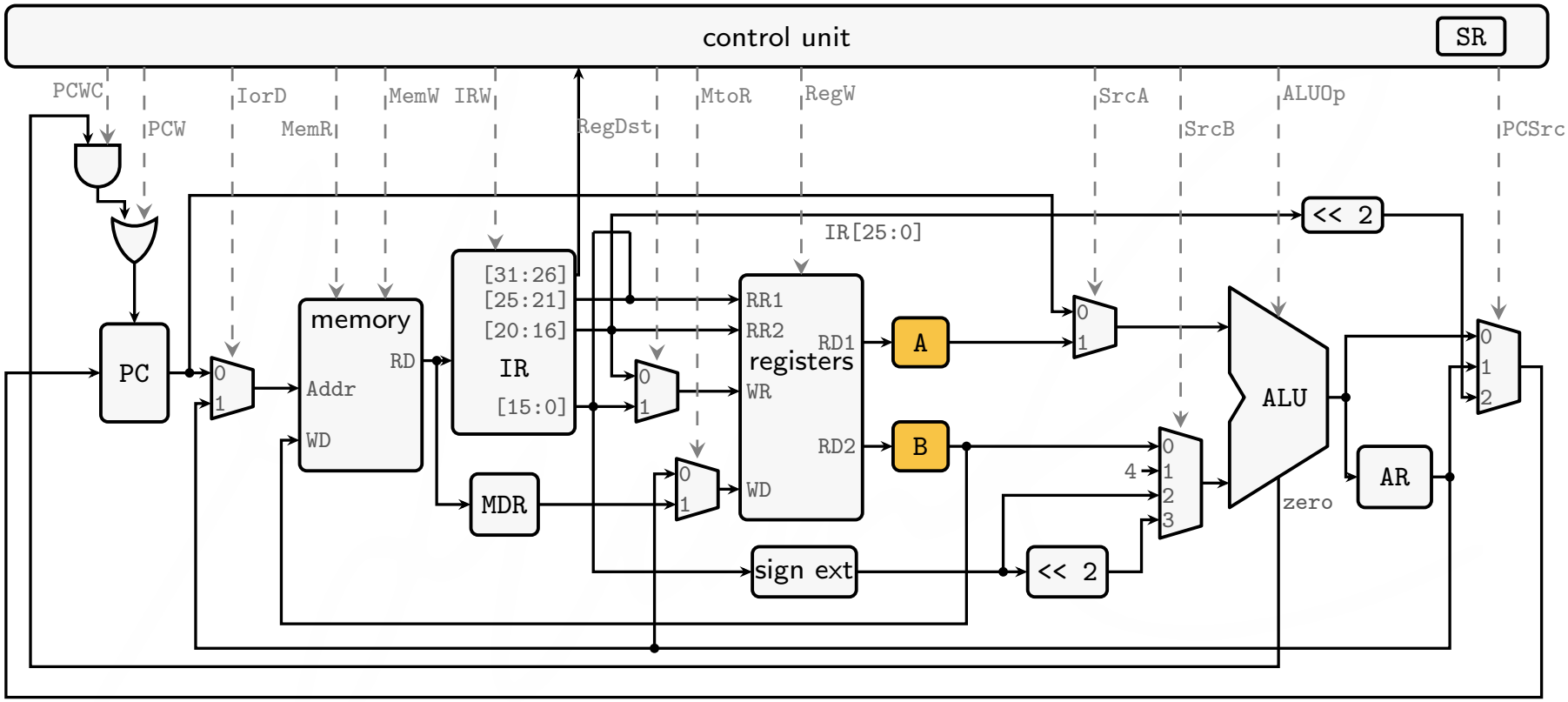
RegDst picks the register named by IR[20:16] or by IR[15:11]

7. The register file reads two registers, and can write one.



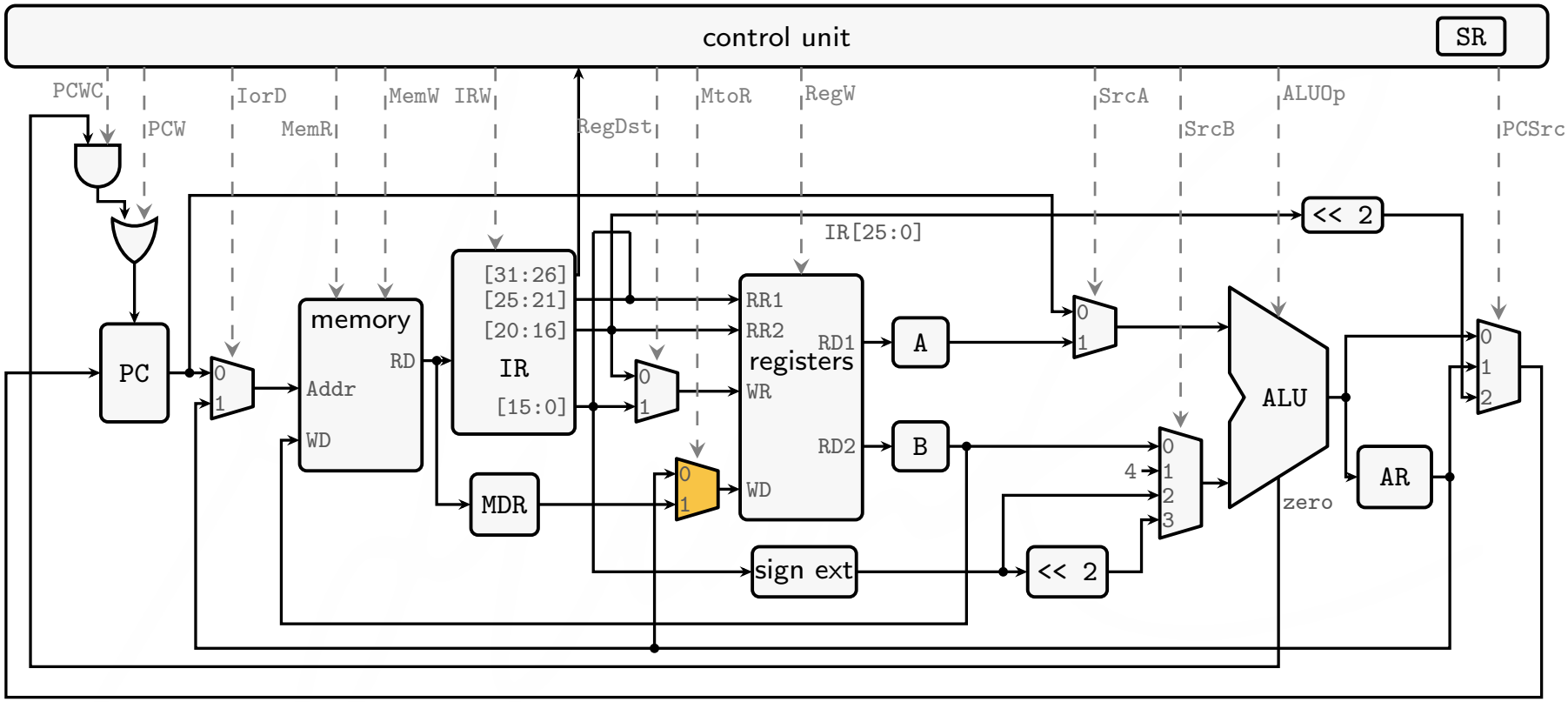
the registers named by `IR[25:21]` and `IR[20:16]` come out on `RD1` and `RD2`; `RegW` writes `WD` into `WR`

8. `A` and `B` hold the two values that were read.



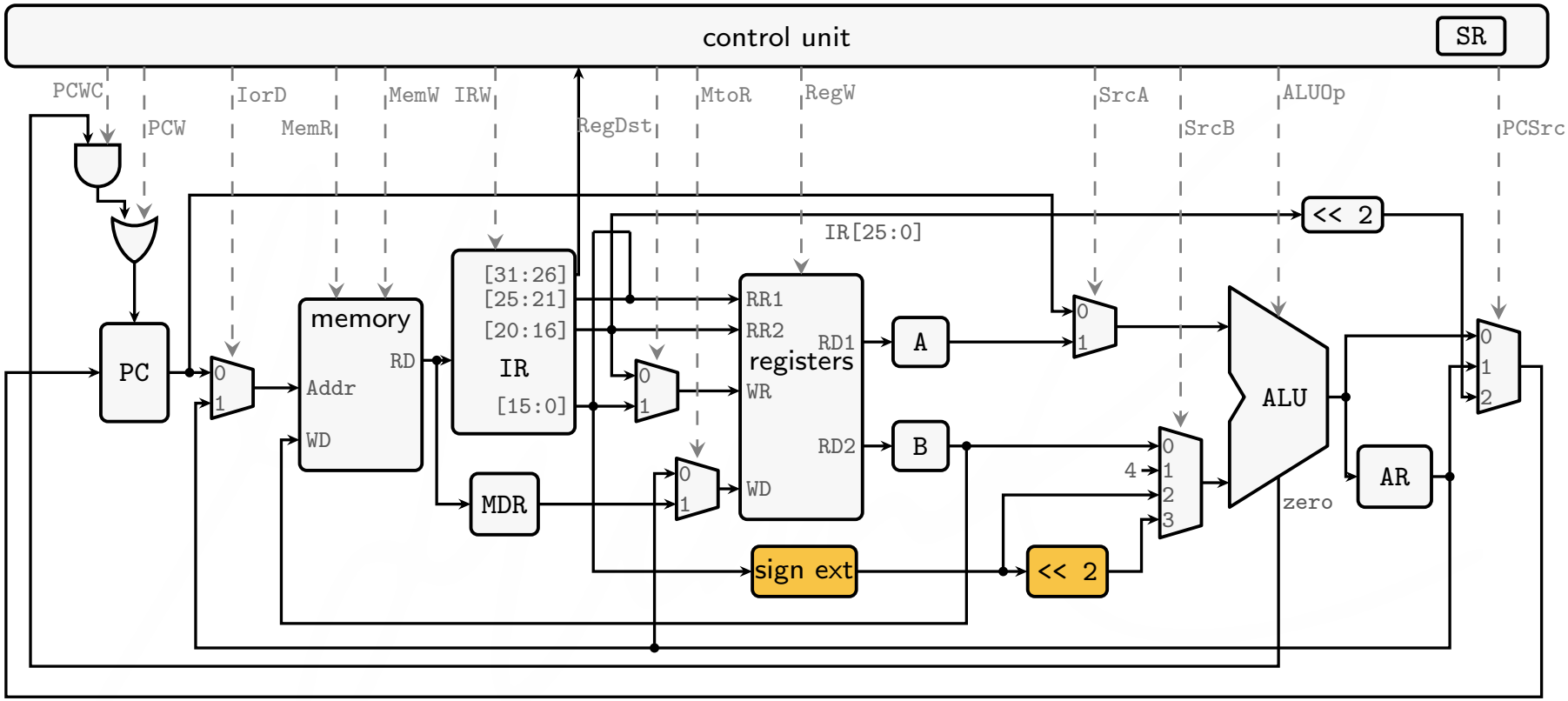
they keep the values steady for the next cycle

9. A mux picks the value written back to a register.



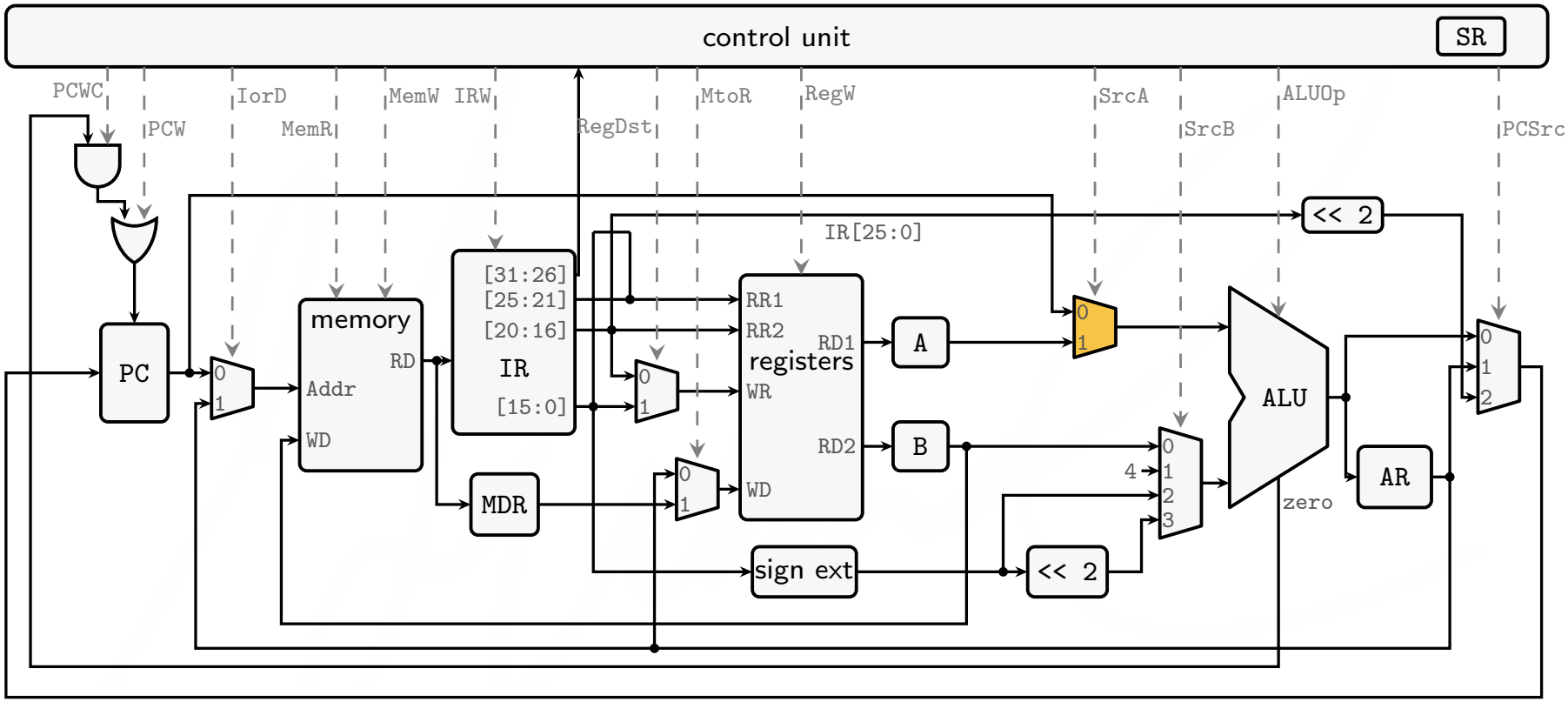
MtoR picks the ALU result in AR, or the loaded word in MDR

10. The constant from the instruction is **sign-extended** to 32 bits.



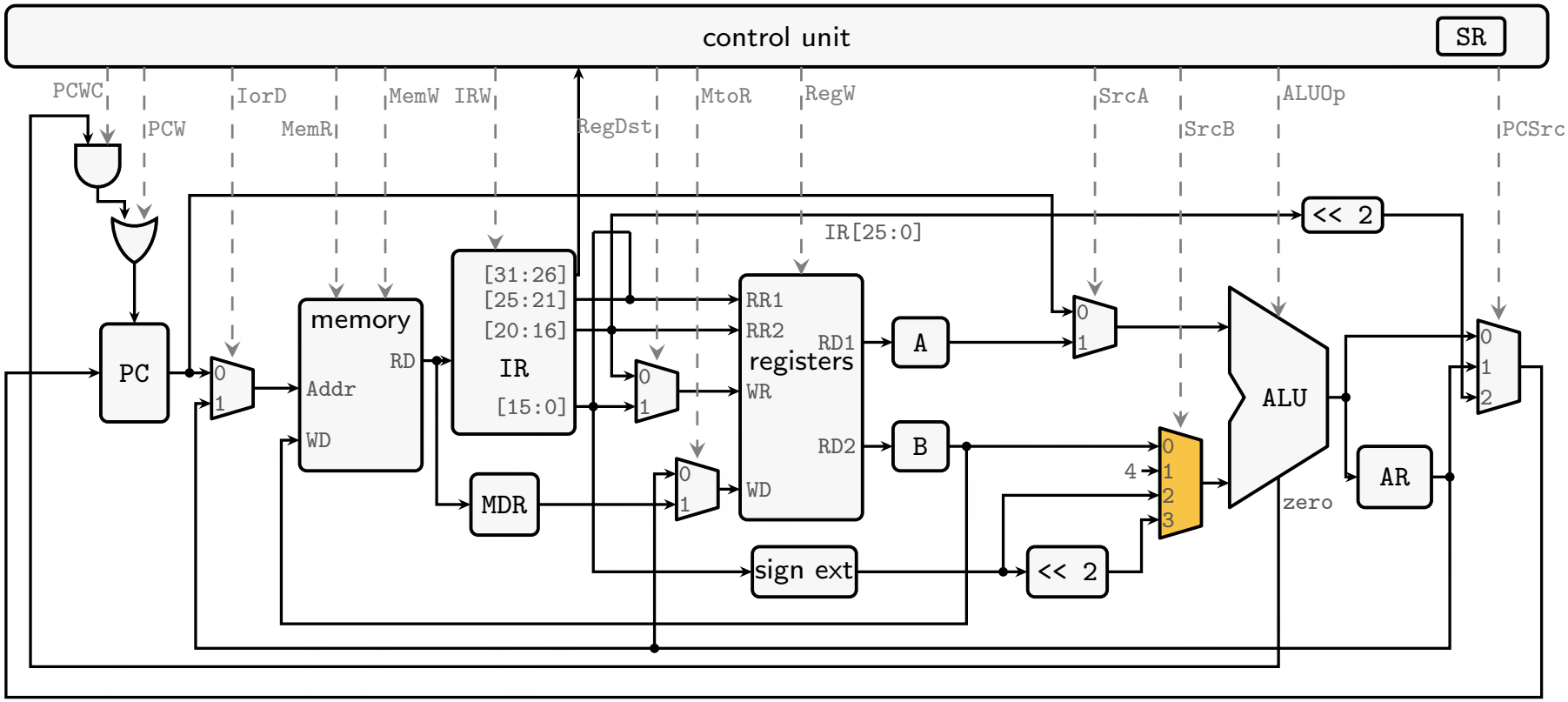
shifting it left by 2 multiplies it by 4, turning a branch offset counted in instructions into one counted in bytes

11. A mux picks the ALU's first input.



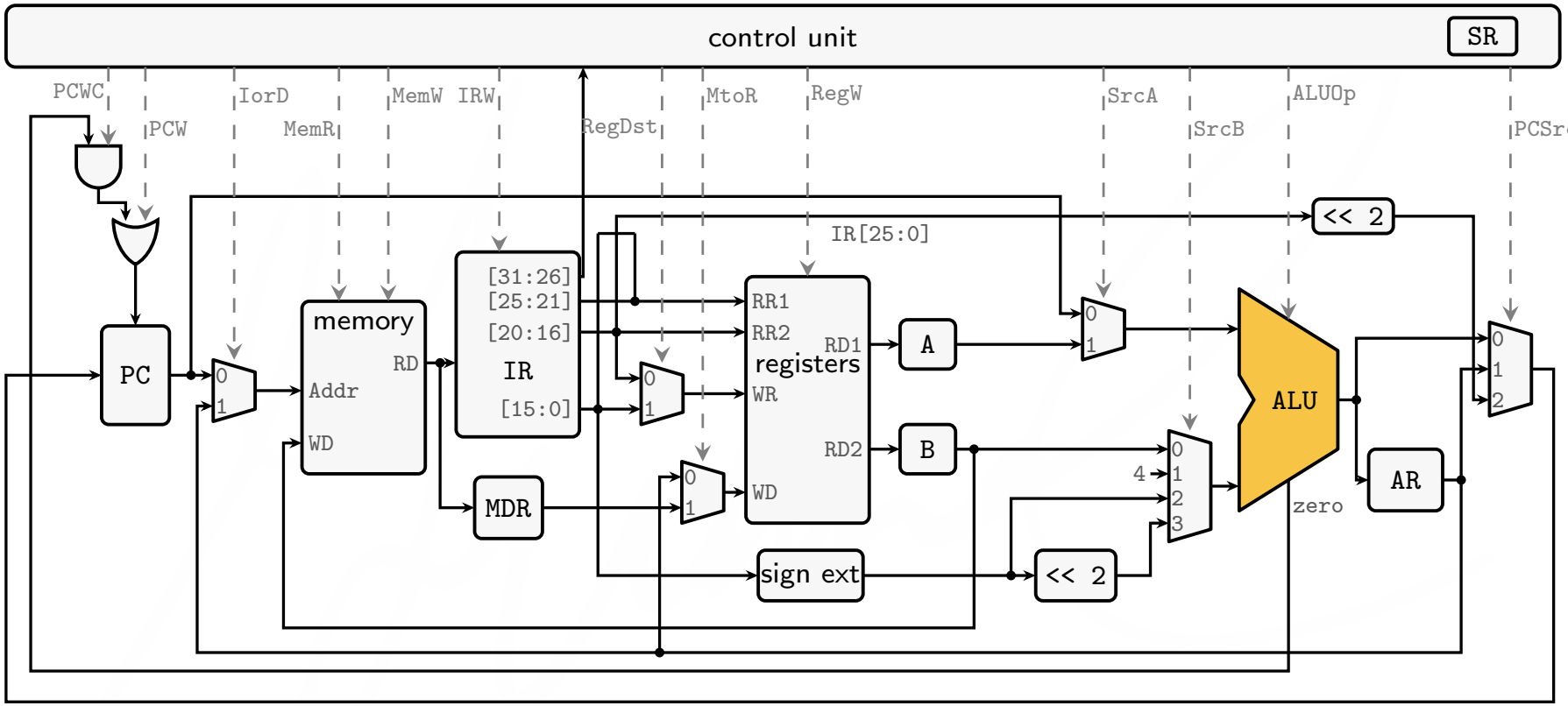
SrcA picks the PC or A

12. A mux picks the ALU's second input.



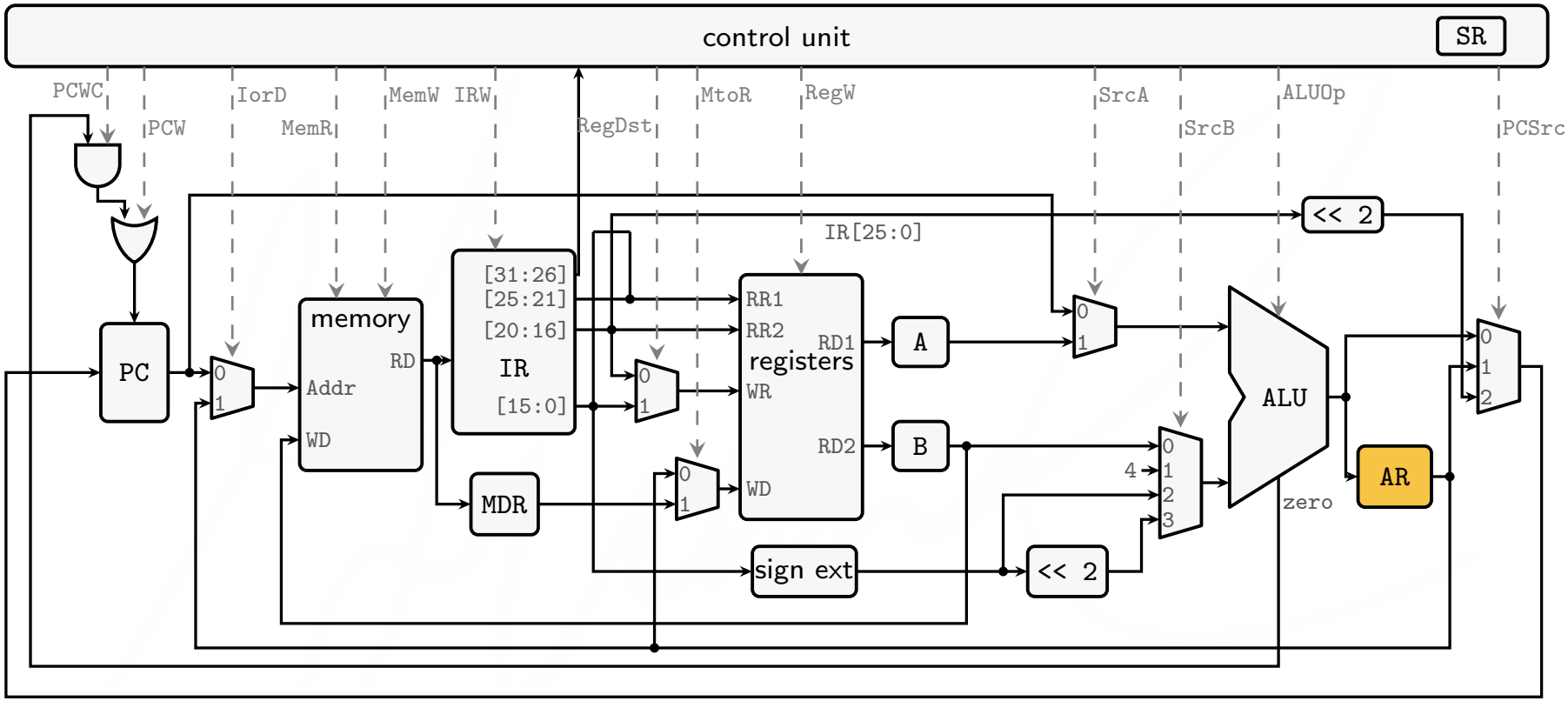
SrcB picks B, the constant 4, the sign-extended constant, or that constant times 4; adding 4 to the PC steps to the next instruction

13. The ALU computes the result.



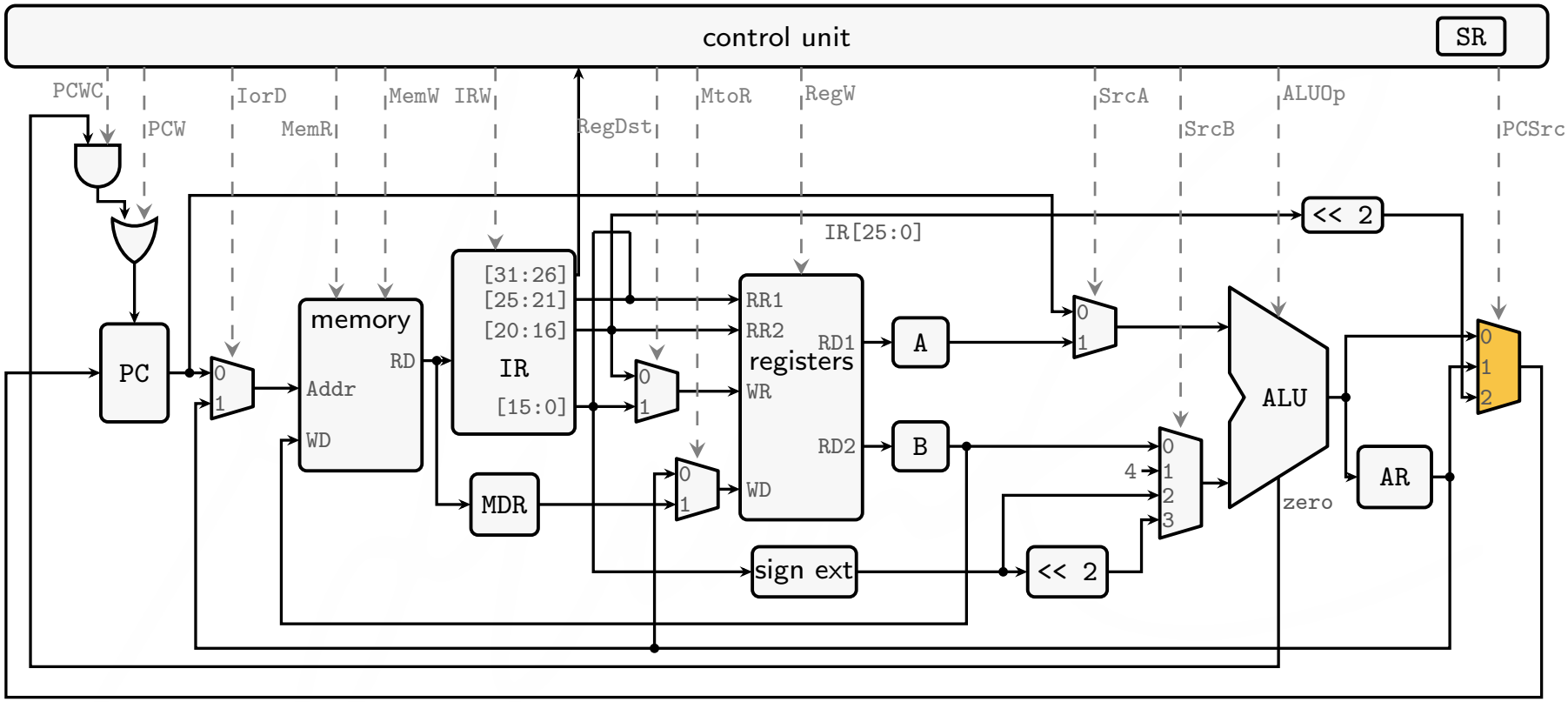
ALUOp chooses the operation; the zero flag says whether the result was 0

14. The **ALU result register** (AR) holds the result.



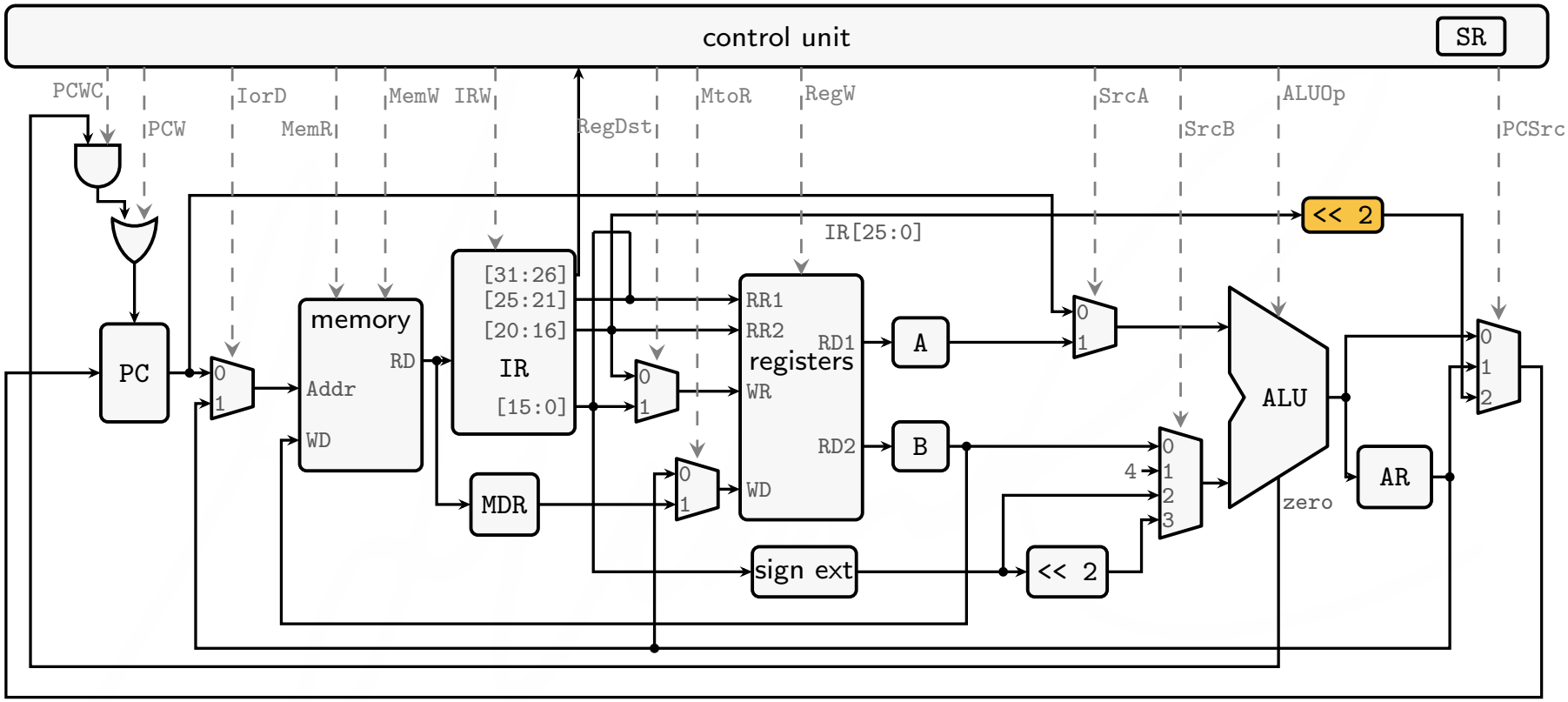
the next cycle can use it as a memory address or write it back to a register

15. A mux picks the next value of the PC.



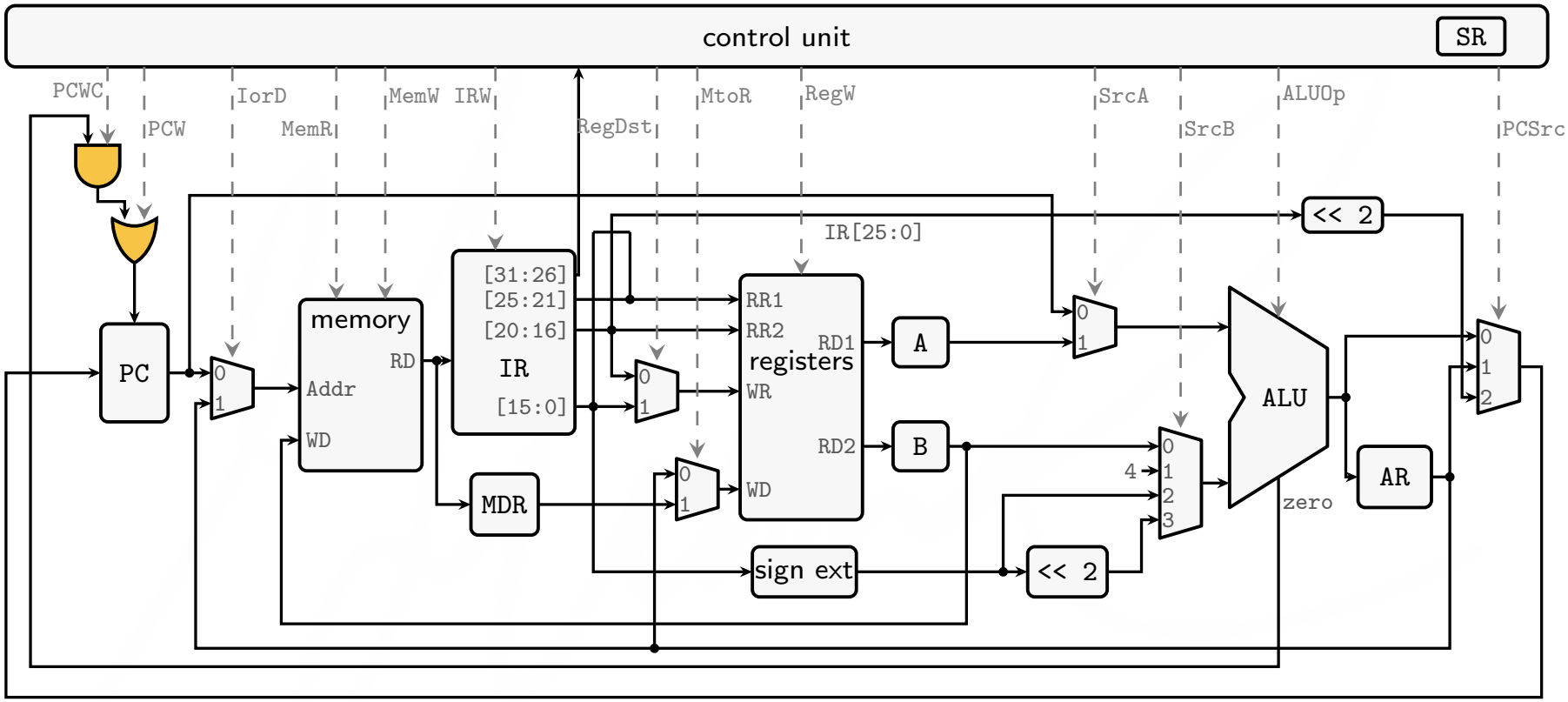
PCSrc picks the ALU result (PC+4), a branch target saved in AR, or a jump address

16. A jump builds its address from the instruction.



the low 26 bits of the instruction, shifted left by 2

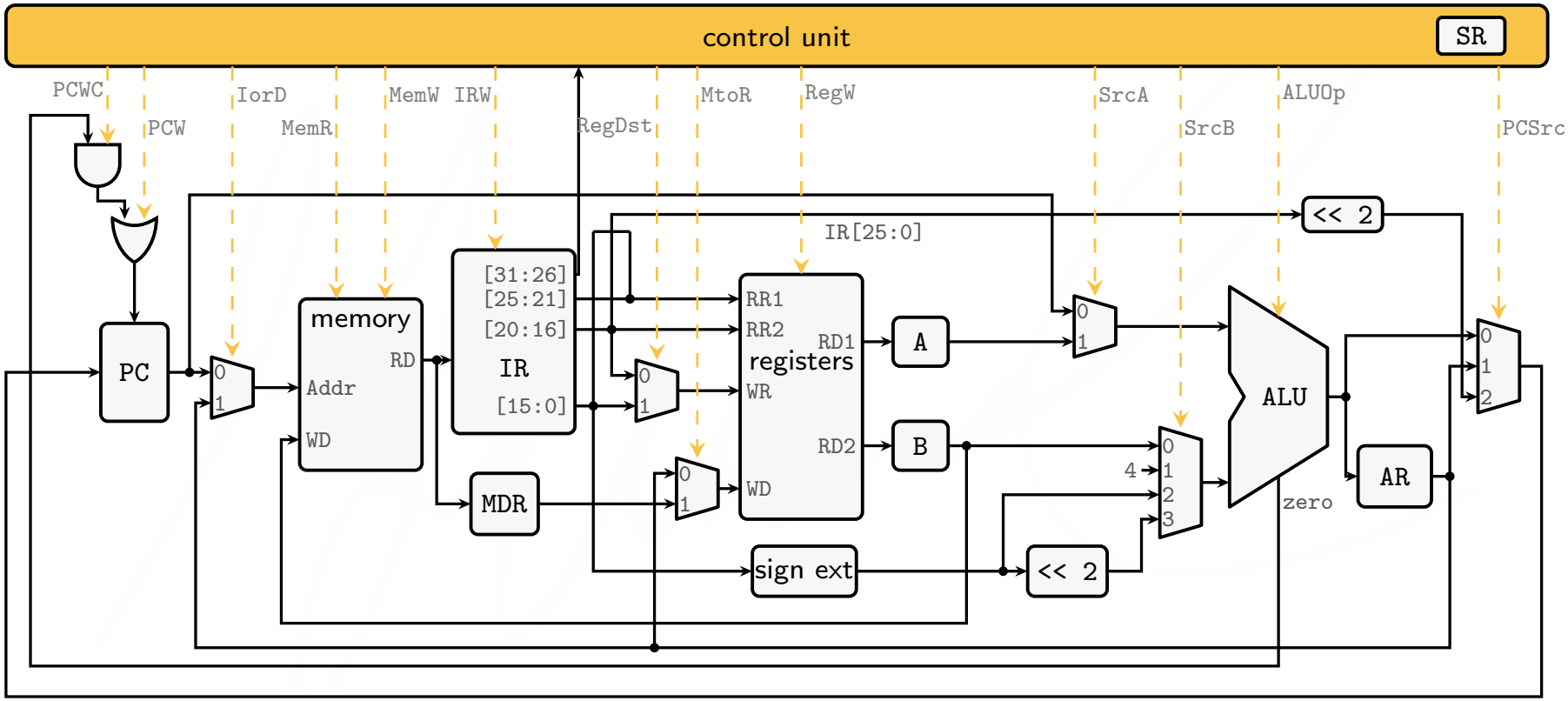
17. Two gates decide when the PC is updated.



it loads whenever PCW is set, or when PCWC is set and zero is 1 (a branch whose two registers are equal)

control

The **control unit** is a finite state machine. It reads the opcode and sets the control signals (dashed) that steer the datapath.



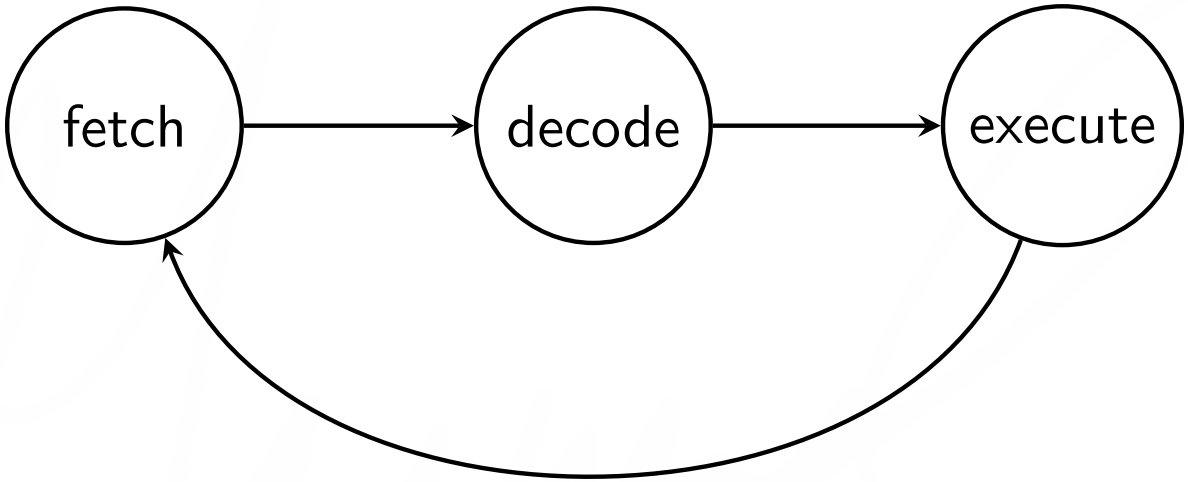
Each control signal sets one part of the datapath.

signal	what it does
`PCW`	load a new value into the `PC`
`PCWC`	load the `PC` only if `zero` is `1`
`IorD`	pick the memory address: `PC` or `AR`
`MemR`, `MemW`	read from or write to memory
`IRW`	load the fetched instruction into the `IR`
`RegDst`	pick the register to write: `IR[20:16]` or `IR[15:11]`
`RegW`	write a register
`MtoR`	pick the write-back value: `AR` or `MDR`
`SrcA`	pick the ALU's first input: `PC` or `A`
`SrcB`	pick the second input: `B`, `4`, the constant, or the constant $\times 4$
`ALUOp`	choose the ALU operation
`PCSrc`	pick the next `PC`: ALU, `AR`, or jump

each state of the machine applies one step of an instruction

the instruction cycle

The control unit repeats the same cycle for every instruction.

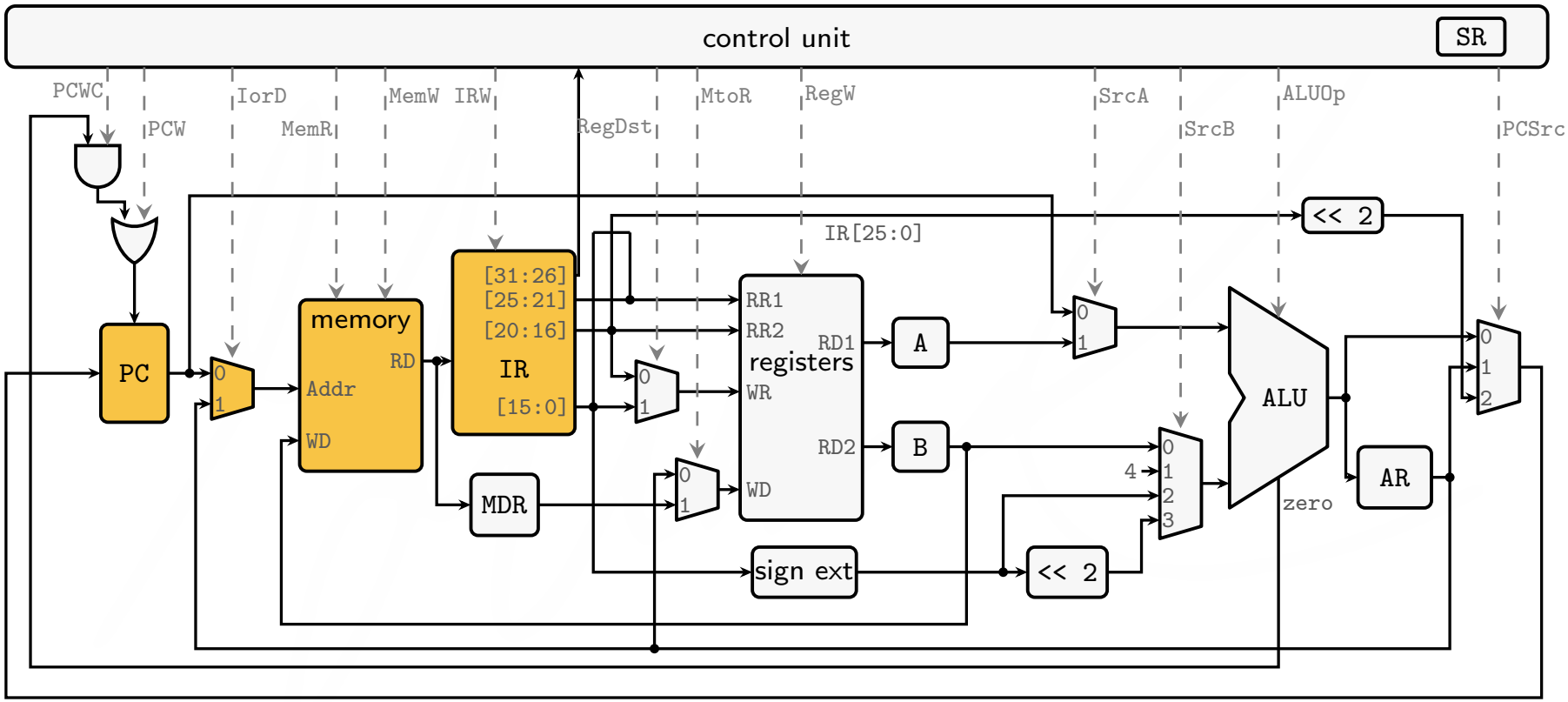


fetch the instruction, decode what it means, then carry it out

- **fetch**: read the instruction at the program counter
- **decode**: work out which operation and operands it names
- **execute**: perform the operation and store the result

one instruction takes several clock cycles, one control state per step

In the **fetch** state, the control unit sets `IorD` to pick the `PC`, `MemR` to read memory, and `IRW` to keep the instruction in the `IR`.



The processor only computes on registers. Special instructions move data between memory and the registers. This is a **load-store** architecture.

- **load** copies a value from memory into a register
- **store** copies a value from a register back to memory

The datapath does the work and the control unit directs it. What each instruction means is defined by the assembly language, which we cover next.